Cross-capacity comparisons for inplace_vector

Document #:	P3698R0 [Latest] [Status]
Date:	2025-05-19
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Charles Hussong <charles.hussong@proton.me>

§ 1 Introduction

`inplace_vector`, which was proposed by [P0843R14] and accepted for C++26, has a missing feature which weakens its API compared to ordinary `vector`: it is not possible to compare instances with different capacities. I believe this should be added before its standardization is completed.

§ 2 The problem

[P0843R14] describes the API of `inplace_vector` as "closely resembl[ing] `std::vector<T, A>`", which indeed is largely the case, including its comparison operators. Like `vector`, `inplace_vector` has comparison operators which look at the current contents, returning a lexicographic ordering following the convention of [container.reqmts].

For `vector`, the current capacity is a dynamic member variable which is ignored for the purposes of comparisons; therefore, the following code works regardless of the internal logic for growing the capacity:

```
#include <cassert>
#include <vector>

int main() {
  std::vector<int> a{1, 2, 3};
  std::vector<int> b{1, 2, 3};
  a.reserve(10);
  b.reserve(100);
  assert(a == b);
}
```

However, for `inplace_vector`, the capacity is a static part of the type itself and the comparison operator is defaulted, so the equivalent code does not work:

```
#include <cassert>
#include <inplace_vector>

int main() {
  std::inplace_vector<int, 10> x{1, 2, 3};
  std::inplace_vector<int, 100> y{1, 2, 3};
  assert(x == y);  // compilation error: decltype(x) not comparable with decltype(y)
}
```

The two vectors with different capacities are distinct types, so the defaulted comparison operator does not support a mixture of them.

This discrepancy with `vector` does not appear in the "Summary of semantic differences with `vector`" section of [P0843R14], leading one to surmise that the intention was for both types to have the same comparison semantics. Private conversations with the [P0843R14] authors confirm that they intended for cross-capacity comparisons to work.

§ 3 Existing practice

The original proposal [P0843R14] cites three library implementations: Boost.Container [1], EASTL [2], and Folly [3]. This paper was prompted by a user question related to an analogous class in ETL [4], so I have included it as well.

Of these,

— Boost claims in its documentation to define the comparisons as free functions templated separately on both capacities [5], however the actual implementation defines the comparisons as non-templated friend functions of a base class which is templated on the capacity [6]

— EASTL defines the comparisons as free functions taking references to an allocator-aware base class, and the template uses the same allocator (thus capacity) for both sides [7]

— Folly defines the comparisons as non-templated member functions [8]

— ETL defines the comparisons in a capacity-erased base class, which is inherited by `vector`s of all capacities [9]

Therefore of the prior art, mixed-capacity comparisons work in ETL, do not work in EASTL and Folly, and Boost claims they work but in fact they do not.

§ 4 Proposed solution

From the current draft wording in [N5008], modify the comparison operators in the listing in section [inplace.vector.overview] to be templated on the capacity:

```
template <size_t M>
constexpr friend bool operator==(const inplace_vector& x, const inplace_vector<T, M>& y);
template <size_t M>
constexpr friend synth-three-way-result<T>
  operator<=>(const inplace_vector& x, const inplace_vector<T, M>& y);
```

After [inplace.vector.erasure], add a new section [containers.sequences.inplace.vector.comparison] with the following contents:

```
template <size_t M>
constexpr friend bool operator==(const inplace_vector& c, const inplace_vector<T, M>& b);
```

Effects: Equivalent to `c == b` defined in [container.reqmts], treating `inplace_vector<T, N>` and `inplace_vector<T, M>` as the same type.

```
template <size_t M>
constexpr friend synth-three-way-result<T>
  operator<=>(const inplace_vector& a, const inplace_vector<T, M>& b);
```

Effects: Equivalent to `a <=> b` defined in [container.opt.reqmts], treating `inplace_vector<T, N>` and `inplace_vector<T, M>` as the same type.

§ 5 What about a generic solution to container comparisons?

There is another paper, [P0805R2], which attempts to generically solve the issue of comparisons between containers which are logically comparable but are not comparable in practice because they are distinct types. Last updated in 2018, this paper would allow comparisons between vectors with different allocator types or arrays with different sizes, among other things. I support this paper as well, but it is much larger in scope and therefore would require more effort and debate to adopt. Since `inplace_vector` is new for C++26, I would like to fix its API before it's released into the wild, and then come back and try to merge the generic solution via an updated [P0805R2] targeting C++29.

The changes proposed by this paper would not conflict with those of [P0805R2], which solves the problem mainly by amending [sequence.reqmts] to be more generic. The API that would be enabled by [P0805R2] would be a superset of the one proposed in this paper, and an updated [P0805R2] could simply widen the templating on the comparison operators proposed here to cover data types as well, and delete the section [containers.sequences.inplace.vector.comparison] which would no longer be needed. This would not break code that works under this paper's wording.

§ 6 Acknowledgements

Thank you to Anthony Williams for help with drafting this paper and Timur Doumler for insight into the intent behind [P0843R14].

§ 7 References

[1] Ion Gaztanaga. Boost.Container.
http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html

[2] Electronic Arts Inc. EA Standard Template Library.
https://github.com/questor/eastl/blob/master/fixed_vector.h#L71

[3] Meta Platforms, Inc. Folly: Facebook Open-source Library.
https://github.com/facebook/folly/blob/main/folly/docs/small_vector.md

[4] John Wellbelove. Embedded Template Library.
https://www.etlcpp.com/vector.html

[5] Ion Gaztanaga. boost/container/static_vector.hpp.
https://www.boost.org/doc/libs/1_59_0/doc/html/boost_container_header_reference.html#header.boost.container.static_vector_hpp

[6] Ion Gaztanaga. boost/container/vector.hpp.
https://www.boost.org/doc/libs/1_59_0/boost/container/vector.hpp

[7] Electronic Arts Inc. eastl/fixed_vector.h.
https://github.com/questor/eastl/blob/624c573fefbd5e9382e726a42590cd8dc9f03916/vector.h#L1999-L2045

[8] Meta Platforms, Inc. folly/container/small_vector.h.
https://github.com/facebook/folly/blob/d17bf897cb5bbf8f07b122a614e8cffdc38edcde/folly/container/small_vector.h#L644-L659

[9] John Wellbelove. etl/vector.h.
https://github.com/ETLCPP/etl/blob/a12dbbd91174e895bdd5492826b0226b9668fc5f/include/etl/vector.h#L1115-L1192

[N5008] Thomas Köppe. 2025-03-15. Working Draft, Programming Languages — C++.
https://wg21.link/n5008

[P0805R2] Marshall Clow. 2018-06-22. Comparing Containers.
https://wg21.link/p0805r2

[P0843R14] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2024-06-26. inplace_vector.
https://wg21.link/p0843r14